

Leadline

A lead management platform, its architecture, permissions and JSON API

A lead management platform for a small sales team — public capture form, role-based team app, pipeline board, and a documented JSON API.

Built for the Digital Heroes Full Stack task (Task A).

1. Live app and tech

<https://leadline-feen.onrender.com>

ROLE	EMAIL	PASSWORD
Admin	admin@leadline.demo	Admin@1234
Member	member@leadline.demo	Member@1234
Member	mia@leadline.demo	Member@1234
Deactivated	former@leadline.demo	Member@1234 (login is refused)

Hosted free, so the first request after ~15 idle minutes takes 30–50 seconds to wake the server.

Tech: Next.js 15 (App Router, TypeScript) · Prisma 6 on SQLite — a local file in development and tests, Turso libSQL in production · zod for validation · bcryptjs + jose for JWT cookie sessions · Tailwind 4 · Vitest · GitHub Actions.

2. Run it locally

BASH

```
cd task_a
npm install
cp .env.example .env      # set AUTH_SECRET to any long random string
npx prisma db push        # create the SQLite schema
npm run db:seed           # load the demo team and 16 sample leads
npm run dev               # http://localhost:3000
```

BASH

```
npm test                  # 40 tests, no external services needed
npm run build              # production build
```

Deploying elsewhere needs three environment variables: `AUTH_SECRET`, plus `TURSO_DATABASE_URL` and `TURSO_AUTH_TOKEN` to point at a hosted database instead of a local file. With those two exported, `npm run db:push:remote && npm run db:seed` prepares the remote database (`prisma db push` only speaks the SQLite file protocol, so `scripts/push-schema.ts` sends the same generated DDL over libSQL).

3. What it does, and why it is built this way

How the code is organized

The split is by *where code runs*, not by file type. Nothing in `server/` is ever sent to the browser, and nothing in `client/` can reach the database — an import in the wrong direction is immediately visible in review.

```
src/
├─ server/          never reaches the browser
│  ├─ db.ts         Prisma client: a local SQLite file, or Turso when TURSO_* is set
│  ├─ http.ts       typed errors → status codes, zod schemas, body and query parsing
│  ├─ auth.ts       password hashing, JWT cookie sessions, role guards, teammate admin
│  ├─ pipeline.ts   pure rules: which stage moves are legal, how a lead is scored
│  └─ leads.ts      lead operations: permissions, persistence, audit trail
├─ client/         the React interface
│  ├─ ui.tsx        status and priority pills, score chips, date and money formatting
│  └─ *.tsx         Board, CaptureForm, LeadActions, LoginForm, QuickAdd, UserForm, Credit
├─ app/            Next.js routing that wires the two together – pages and route handlers
└─ middleware.ts   edge check on the session cookie, before a page renders
```

`pipeline.ts` imports nothing — no database, no framework, no request object. The stage machine and the scoring formula are plain functions over plain values, which is why the rules are easy to read, easy to test, and impossible to bypass by calling a different code path.

Route handlers stay four or five lines each: work out who is calling, hand the request to a server function, return what comes back. All the judgement lives one layer down, so pages and endpoints cannot drift apart.

The pipeline is a board with rules

The dashboard is a drag-and-drop Kanban board over six stages: New → Contacted → Qualified → Proposal → Won / Lost. Dragging a card fires the same `PATCH /api/leads/:id` any API client would call, so the rules live in one place and the UI cannot cheat them:

- Forward moves and one step back are allowed; skipping stages is not.
- Won and Lost are terminal for members. Only an admin can reopen them.
- Moving to Lost requires a reason, which is cleared if the lead is later reopened.

A rejected move returns `422` with the stages that *were* allowed, and the card snaps back with that message. Funnel numbers stay honest because nobody can teleport a lead from New to Won.

Two roles, enforced on both sides

Route handlers only work out *who* is calling. Every domain function then takes that caller and decides *what they may do*, throwing typed errors that one wrapper turns into HTTP codes. Server components call the same functions as the API, so a page can never be more permissive than an endpoint.

ACTION	ADMIN	MEMBER
View all leads, notes, activity	yes	yes
Create leads	yes, assign anyone	yes, unassigned or self
Edit / move status / add notes	any lead	only leads assigned to them
Assign, delete leads	yes	403
Reopen a Won or Lost lead	yes	422
Manage teammates	yes	403

On the client, middleware redirects guests to login and keeps members out of `/team`, admin-only controls are not rendered for members, and closed cards are not draggable. That is convenience — the server refuses the same requests regardless.

Sessions are a 7-day JWT in an `HttpOnly`, `SameSite=Lax` cookie. Middleware checks the signature at the edge for redirects; API requests re-read the user from the database every time, so deactivating someone revokes their access immediately even though their token is still signed and unexpired.

The activity trail cannot lie

Every mutation writes its `Activity` row inside the same transaction as the change itself, so the history and the data can never disagree. Rows are append-only, and notes cannot be edited or deleted. Names are copied into the activity metadata at write time, so the story still reads correctly after someone is renamed or deactivated.

That is also why users are deactivated rather than deleted: their notes and actions keep their author. Leads carry a denormalized `lastActivityAt` so "stalest first" is a sort, not a join.

Leads arrive scored, and can arrive from anywhere

Each write recomputes a 0–100 score from deal value, priority, source, and how recent the lead is. The board sorts hottest-first and flags anything above 60 with a flame, so the morning question "who do I call?" has an answer without reading every card.

`GET /embed.js` returns a loader that drops the capture form onto any website as an iframe:

HTML

```
<script src="https://leadline-feen.onrender.com/embed.js" async></script>
```

One tag on a client's site and their inquiries land in this pipeline, scored, with an audit trail — which is the difference between a lead form and a lead platform. The public endpoint also carries a honeypot field: humans never see it, and a bot that fills it gets `201` while nothing is stored.

Design

A sales pipeline is really a database with views, so the interface borrows Notion's vocabulary: a sidebar workspace, page emoji and title, Board and Table tabs, filter chips, soft pastel tag pills, and lead pages laid out as documents with a property grid, note blocks, and a quiet activity timeline. Status colours are declared once and reused everywhere so the product reads as one thing.

The API

Every screen is built on it. Cookie auth; bodies and responses are JSON.

METHOD	ENDPOINT	WHO	NOTES
POST	<code>/api/public/leads</code>	anyone	capture form; honeypot; <code>422</code> with per-field errors
POST	<code>/api/auth/login</code> · <code>/api/auth/logout</code>	anyone	<code>401</code> on bad credentials or a deactivated user
GET	<code>/api/auth/me</code>	signed in	current user
GET	<code>/api/leads</code>	any role	paginated and filtered — see below
POST	<code>/api/leads</code>	any role	members may only self-assign
GET	<code>/api/leads/:id</code>	any role	with assignee and notes
PATCH	<code>/api/leads/:id</code>	assignee or admin	fields and status; <code>422</code> on an illegal move
DELETE	<code>/api/leads/:id</code>	admin	<code>204</code>
PATCH	<code>/api/leads/:id/assign</code>	admin	<code>{ "assignedToId": id null }</code>
GET, POST	<code>/api/leads/:id/notes</code>	admin or assignee to post	timestamped, immutable
GET	<code>/api/leads/:id/activities</code>	any role	the audit trail
GET	<code>/api/leads/export</code>	any role	CSV, same filters as the list
GET, POST	<code>/api/users</code> , PATCH <code>/api/users/:id</code>	admin	invite, list, deactivate

List parameters: `page`, `pageSize` (max 100), `status`, `priority`, `assignedTo` (an id or `unassigned`), `q` (name, email, company), `sort` (`createdAt`, `lastActivityAt`, `value`, `score`), `order`. Anything invalid is a `400`.

BASH

```
curl -c jar.txt -X POST https://leadline-feen.onrender.com/api/auth/login \  
  -H 'content-type: application/json' \  
  -d '{"email":"admin@leadline.demo","password":"Admin@1234"}'  
  
curl -b jar.txt 'https://leadline-feen.onrender.com/api/leads?  
status=QUALIFIED&sort=score&order=desc'
```

```
{
  "data": [{ "id": "...", "name": "Sara Fernandes", "status": "QUALIFIED", "score": 86 }],
  "meta": { "page": 1, "pageSize": 20, "total": 3, "totalPages": 1 }
}
```

Codes used: `200` `201` `204` · `400` bad query or malformed JSON · `401` not signed in · `403` not allowed · `404` missing · `409` duplicate email · `422` failed validation or a broken domain rule, with `details` naming what went wrong.

Data model

`User` (role, active flag) · `Lead` (contact details, source, status, priority, value, score, follow-up date, lost reason, assignee) · `Note` (immutable, authored) · `Activity` (append-only, typed, with JSON metadata). Notes and activities cascade with their lead; deleting a user only nulls their assignments. Indexes cover the ways the board actually reads: by status and assignee, by recency, by score.

Tests

`npm test` runs 40 cases against a throwaway SQLite file, calling the real route handlers with real login cookies — no server and no network needed.

- **Auth rules** — wrong password, deactivated user, missing and forged cookies, and the full member-versus-admin matrix across leads, users, assignment, deletion, and export.
- **Public capture** — a valid submission creates a New web-form lead with its activity row and a score; invalid input returns field errors; the honeypot stores nothing.
- **Lead lifecycle** — create, assign, advance, note, then assert the four-event trail in order; stage-skips and missing Lost reasons rejected; members blocked from reopening while admins can; filters, pagination arithmetic, and invalid parameters.

Every push runs the same suite plus a production build in GitHub Actions.

Choices and limits worth naming

SQLite was chosen so the test suite needs no services and the same engine ships to production through Turso; the cost is no native enum type, so enum-like columns are strings pinned by zod at the edge and by the service layer underneath. Logging out clears the cookie but the JWT stays valid until it expires — the usual stateless-session trade. Two inquiries from one email are two leads, because merging is a judgement call a salesperson should make. Spam protection is a honeypot rather than a rate limiter. Deletes are admin-only and cascade; notes never change.

Where AI was used

I used Claude as a pair programmer: sketching the architecture, drafting route handlers and tests, and arguing with me about choices like SQLite versus Postgres and how strict the stage rules should be. I set the product direction — the board, the scoring, the embeddable widget, the Notion-style interface — reviewed and reshaped what came back, and verified every flow by hand against the running app.